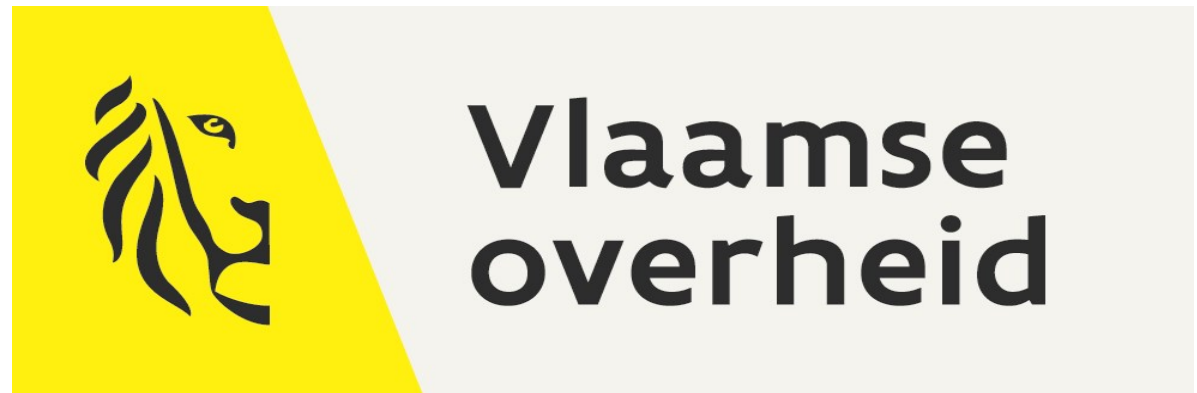


WaterInfo Vlaanderen **Use case**

Shehabeldeen Abdelfatah



Introduction

The Waterinfo.be use case presents a transformative approach to managing real-time hydrological sensor data by converting traditional CSV and JSON outputs into semantically enriched Linked Data Event Streams (LDES).

This initiative enhances data interoperability, accessibility, and reusability, enabling more efficient integration with external datasets and real-time event-driven applications.

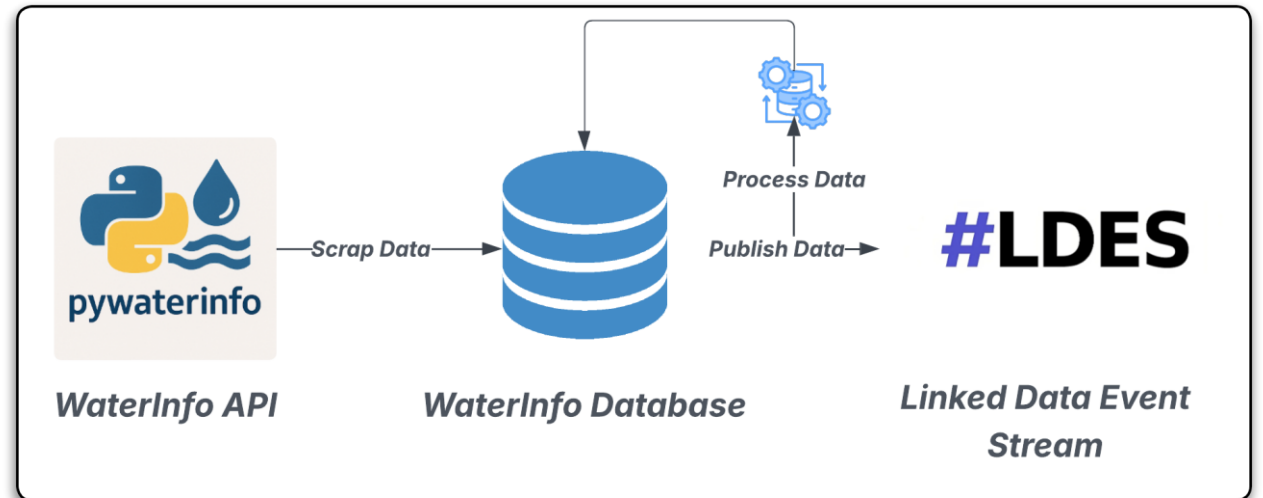
By leveraging standards like SSN/SOSA, SHACL, and DCAT, the project aims to support scalable, low-latency processing of water-related observations, crucial for applications such as flood forecasting and water quality monitoring.

Challenges with the current data format

- Heavy client-side parsing and maintenance burdens
- Batch-only updates that introduce latency
- Limited semantic context and opaque metadata
- Difficulties in federating with other datasets
- No fine-grained provenance or versioning
- Complicated query logic for real-time or windowed analysis

Objectives

- **Standardize** semantically rich metadata, ensuring uniform interpretation.
- **Integrate** seamlessly with external datasets (spatial, regulatory, historical).
- **Enable** real-time, event-driven processing (alerts, continuous analytics).
- **Enhance** discoverability via SPARQL queries, catalogs, and linked-data registries.
- **Facilitate** complex temporal and federated queries for deeper insights.
- **Future-proof** your data schema and encourage modular pipeline architectures.
- **Maintain** precise provenance and governance over individual sensor events.
- **Tap into** a broad ecosystem of semantic-web and stream-processing tools.



What is Waterinfo Vlaanderen ?

- Waterinfo Vlaanderen is the official online portal of the Flemish government for real-time water-related data and forecasts.
- It is managed by the Vlaamse Milieumaatschappij (VMM).
- It provides comprehensive information on water levels, tides, rainfall, droughts, and flood risks throughout Flanders.

Key Features of Waterinfo Vlaanderen

- Real-Time Water Levels and Flood Forecasts: Access current water levels and 48-hour flood predictions for rivers and streams across Flanders.
- Rainfall and Drought Monitoring: Utilize detailed rainfall radar images and data from pluviometers to assess recent and predicted precipitation.
- Tide Information: Provides tidal forecasts and digital tide books.
- Flood Risk Maps and Watercourse Data: Offers access to flood risk maps.

Access to Data

- There exist two libraries, both in Python and R, that facilitate fetching data from Waterinfo.
- The libraries' development and maintenance is supported by Fluves. Part of the initial development of this package has been supported by VITO.
- We also requested tokenized access to obtain more data.

FLUVES

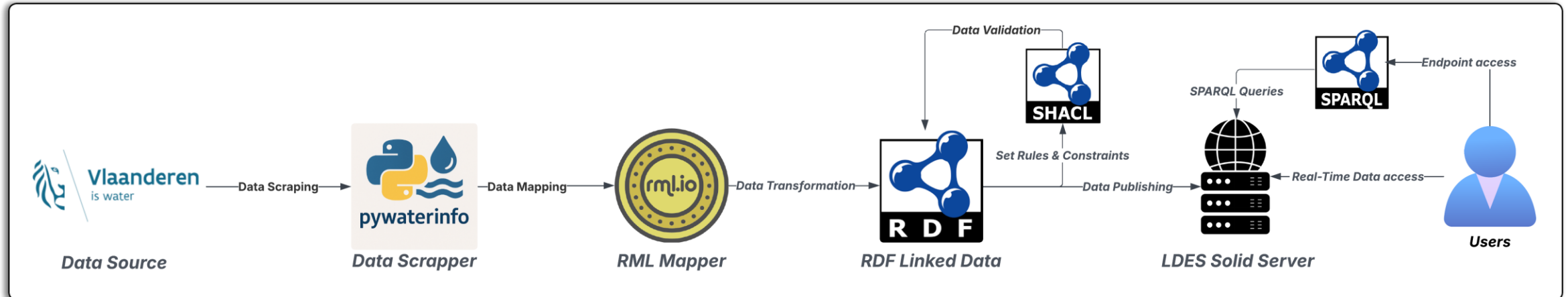


How does the data look like?

- Data is provided from two sources, which are VMM (Vlaamse Milieumaatschappij) and HIC (Hydrologisch Informatiecentrum).
- It contains information on both sensors and observations.
- For sensors, it contains information such as station location, address, etc.
- For observations, it contains information such as timestamp, reading value, unit measure, etc.

Proposed data pipeline

1. Data scraper: fetch data from VMM and HIC. Data is available up to 5 years, but with usage limits.
2. Data pre-processing: adjust attributes fetched to be URI-compliant.
3. Mapping rules: transform data from discrete CSV readings into a relational form. Data is mapped using SSN SOSA
4. Data post-processing: designed to be a generic module that modifies RDF files to add back prefixes
5. Rules and Constraints: Rules are written in the form of SHACL shapes and are validated against data entries.
6. LDES conversion: convert RDF-compliant data to LDES along with adding TREE hypermedia.
7. SPARQL Query: a standalone module designed to execute SPARQL queries on knowledge graphs.



Proposed data pipeline

1. Data scraper

The first step is to fetch data from Waterinfo.be. The data is divided between VMM (Vlaamse Milieumaatschappij) and HIC (Hydrologisch Informatiecentrum) with separate APIs.

Fetching data from VMM or HIC will return a group list of stations, each with its own unique ID. Using those IDs, we can fetch information about the stations themselves along with information on the sensors. Each sensor has a unique Id and can be used to fetch historical data. Data is available up to 5 years but with usage limits.

The data scraper has been built as a standalone module that can be used separately. Please refer for Waterinfo.be for usage limits and API keys. The module can be found separately through this link: <https://github.com/ShehabEldeenAyman/waterinfo-scrap>

Proposed data pipeline

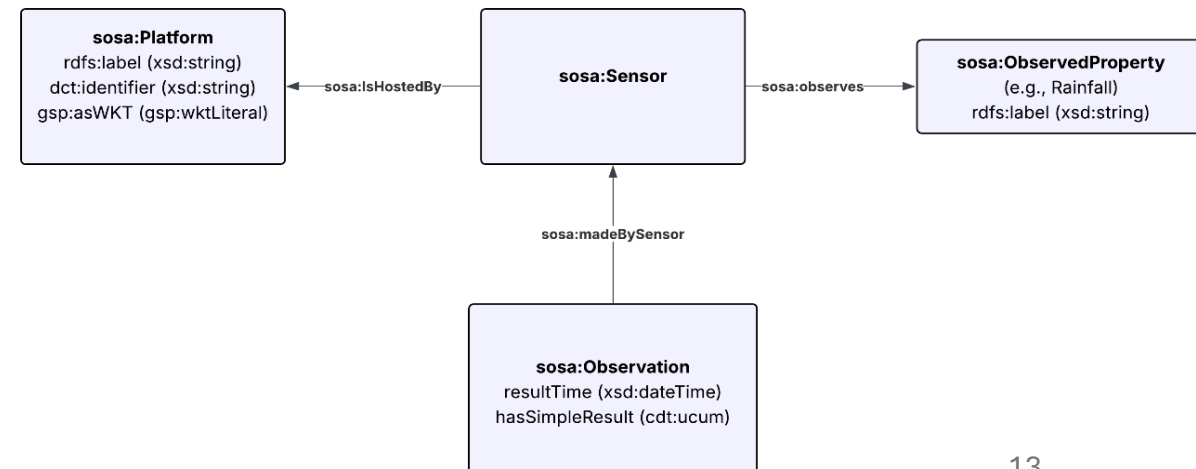
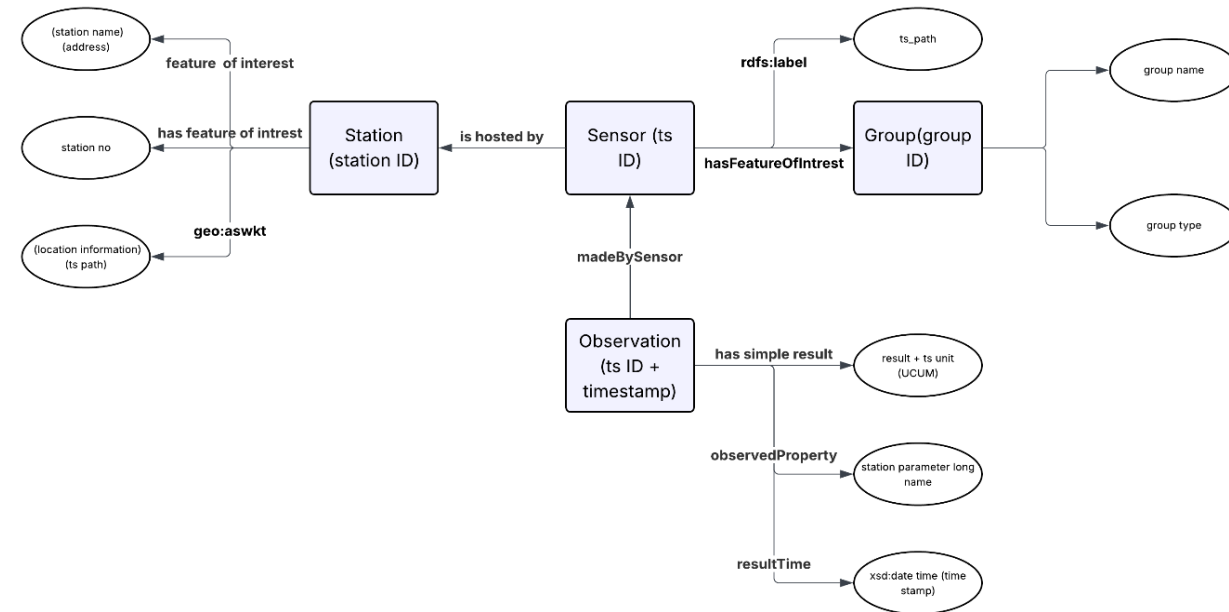
2. Data pre-processing

The data pre-processing module is designed to adjust attributes fetched from Waterinfo.be to be URI compliant. Names that include spaces or unsupported characters are modified. Also, unit measurements are changed to be UCUM “Unified Code for Units of Measure” compliant.

Proposed data pipeline

3. Mapping rules

Mapping rules are implemented to transform data from discrete CSV readings into a relational form. Data was mapped using SSN SOSA along with other standards in order to gain a more semantic understanding of the data. Also, to have a better linkage between data entities. Mapping was achieved by using RML Mapper and RML Streamer modules to transform CSV to RDF.



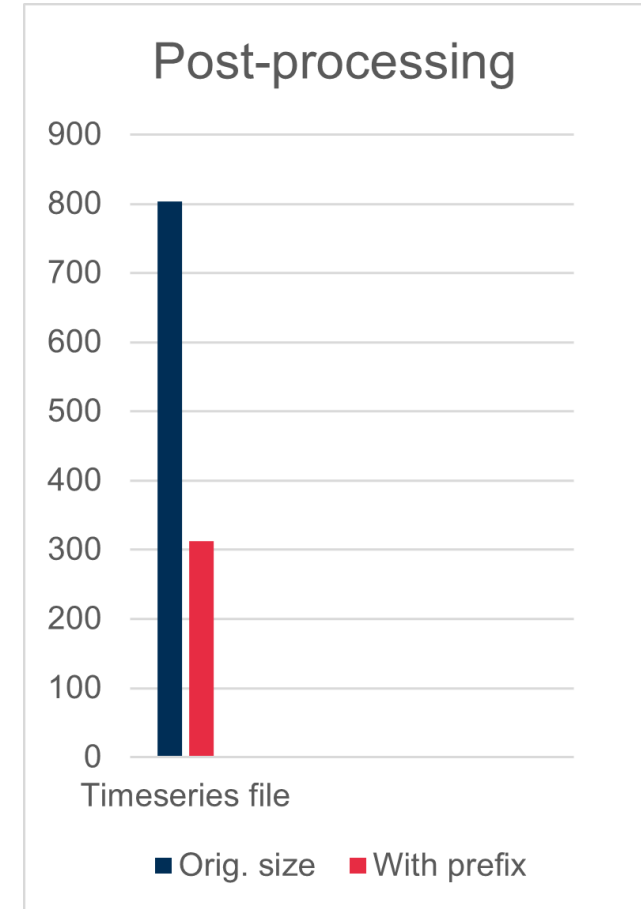
Proposed data pipeline

4. Data post-processing

Data post-processing module is designed to be a generic module that modifies RDF files to add back prefixes, resulting in a huge reduction in file size which in return reduces the load on computational and storage resources.

With a timeseries file of 591349 records, the prefix post-processing technique was able to reduce the file size to 39% of its original size.

The data post-processing module was designed to be generic, and it can work on any RDF-compliant data. The module has been published separately and can be found on: <https://github.com/ShehabEldeenAyman/rdf-prefix-binder>



Proposed data pipeline

5. Rules and Constraints

Rules and constraints are designed to make sure that the data added to the pre-determined model is compliant. Rules are written in the form of SHACL shapes and are validated against data entries. Rules are flexible in the sense that any rule that needs to be enforced on the data can be specified.

Proposed data pipeline

6. LDES conversion

The LDES conversion module is in development as a standalone module. Its goal is to convert RDF-compliant data to LDES along with adding TREE hypermedia. Currently, the module is in early development with focus on sensory observations coming from waterinfo.be. Later, it will take a more general form to be compatible with all kinds of star pattern observations.

```
10 ex:SaltSensor1 a sosa:Sensor ;
11     rdfs:label "Salt Concentration Sensor" ;
12     sosa:observes ex:SaltConcentration .
13
14 ex:WaterSampleLocation1 a sosa:FeatureOfInterest ;
15     rdfs:label "Water Sample Location 1" .
16
17 # Observation 1
18 ex:Obs1 a sosa:Observation ;
19     sosa:madeBySensor ex:SaltSensor1 ;
20     sosa:hasFeatureOfInterest ex:WaterSampleLocation1 ;
21     sosa:observedProperty ex:SaltConcentration ;
22     sosa:resultTime "2025-05-15T08:00:00Z"^^xsd:dateTime ;
23     sosa:hasSimpleResult "3500"^^xsd:double ;
24     qudt:unit unit:PPM .
```

```
18 # 2. Define the event stream
19
20 <#EventStream> a ldes:EventStream ;
21     rdfs:label "Salt Concentration LDES Stream" ;
22     ldes:timestampPath sosa:resultTime ;
23     # order events by the sosa:resultTime property:
24     tree:member ex:Obs1, ex:Obs2, ex:Obs3 . #only include members of the first page
25
26
27 # 5. The actual observations (same as before)
28 ex:Obs1 a sosa:Observation ;
29     sosa:madeBySensor ex:SaltSensor1 ;
30     sosa:hasFeatureOfInterest ex:WaterSampleLocation1 ;
31     sosa:observedProperty ex:SaltConcentration ;
32     sosa:resultTime "2025-05-15T08:00:00Z"^^xsd:dateTime ;
33     sosa:hasSimpleResult "3500"^^xsd:double ;
34     qudt:unit unit:PPM .
```

Proposed data pipeline

7. SPARQL Query

The SPARQL query executor is designed as a standalone module that executes queries. The executor takes a file with a query text along with a directory of the data model. The executor checks the query for errors. Once passed, a text file is generated with the result of the query.

Link to
Repository

